

MySQL

What are Aggregation Functions?

Definition:

Aggregation functions perform calculations on multiple rows of data and return a single result.

They are mainly used to:

- Calculate totals
- Find averages
- Count records
- Find minimum & maximum values

```
SELECT SUM(Marks) AS Total_Marks  
FROM Students;
```

Table: Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2

SELECT SUM(amount) AS total_sales
FROM Orders;

total_sales

13350

```
SELECT AVG(Marks) AS Average_Marks  
FROM Students;
```

Table: Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2



```
SELECT customer_id, AVG(amount) AS average_spends  
FROM Orders  
GROUP BY customer_id;
```



customer_id	average_spends
1	400
2	250
3	12000
4	350

```
SELECT COUNT(*) AS Total_Students  
FROM Students;
```

```
SELECT MIN(Marks) AS Lowest_Marks,  
       MAX(Marks) AS Highest_Marks  
FROM Students;
```


What is GROUP BY?

GROUP BY is used to:

- Divide rows into groups
- Apply aggregation function to each group

Calculate total marks by department

```
SELECT Department, SUM(Marks) AS Total_Marks  
FROM Students  
GROUP BY Department;
```

GROUP BY with AVG()

```
SELECT Department, AVG(Marks) AS Avg_Marks  
FROM Students  
GROUP BY Department;
```

HAVING Clause

HAVING is used to:

- Filter grouped results
- Works with GROUP BY

 Difference:

- WHERE → Filters rows
- HAVING → Filters groups

```
SELECT Department, SUM(Marks) AS Total_Marks  
FROM Students  
GROUP BY Department  
HAVING SUM(Marks) > 200;
```

What is a Primary Key?

A **Primary Key** is a column (or combination of columns) that uniquely identifies each row in a table.

Rules:

- Must contain **unique values**
- Cannot contain **NULL**
- Only **one primary key** per table

```
CREATE TABLE Students (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

What is a Foreign Key?

A Foreign Key is a column that creates a relationship between two tables.

It refers to the Primary Key of another table.

Rules:

- **Can contain duplicate values**
- **Can contain NULL (depending on constraint)**
- **Multiple foreign keys allowed**


```
CREATE TABLE Orders (  
    Order_ID INT PRIMARY KEY,  
    Student_ID INT,  
    Course VARCHAR(50),  
    FOREIGN KEY (Student_ID) REFERENCES Students(Student_ID)  
);
```

Relationship Between Tables

Students Table (Parent Table)

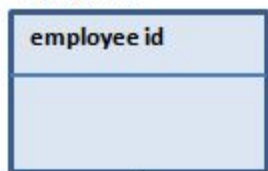


Orders Table (Child Table)

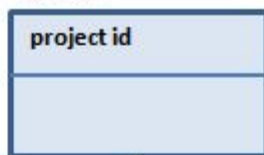
Foreign Key ensures:

- You cannot insert `Student_ID = 10` in Orders if it doesn't exist in Students.
- Maintains **referential integrity**

EMPLOYEE



PROJECT



is assigned to

has as project members

PROJECT EMPLOYEE



Subqueries

A **subquery** is a query written inside another query.

It is also called a **nested query**.

Subqueries can be used in:

-  **SELECT** clause
-  **FROM** clause
-  **WHERE** clause

Subquery in **WHERE** Clause

Used to filter records based on another query result.

Example: Get students who scored above average marks

```
SELECT name, marks
FROM skill
WHERE marks > (
    SELECT AVG(marks)
    FROM skill
);
```

Inner query calculates average marks.

Outer query selects students whose marks are greater than that average.

Subquery in SELECT Clause

👉 Used to return additional calculated values in each row.

Example: Show each student's marks and overall average marks

```
SELECT
    name,
    marks,
    (SELECT AVG(marks) FROM skill) AS average_marks
FROM skill;
```

Each row will show:

- Student name
- Student marks
- Overall average (same value in every row)

Subquery in **FROM** Clause

Used as a temporary table (derived table).

Example: Get average marks per course and show only courses with average > 75

Inner query creates a temporary table with course-wise averages.

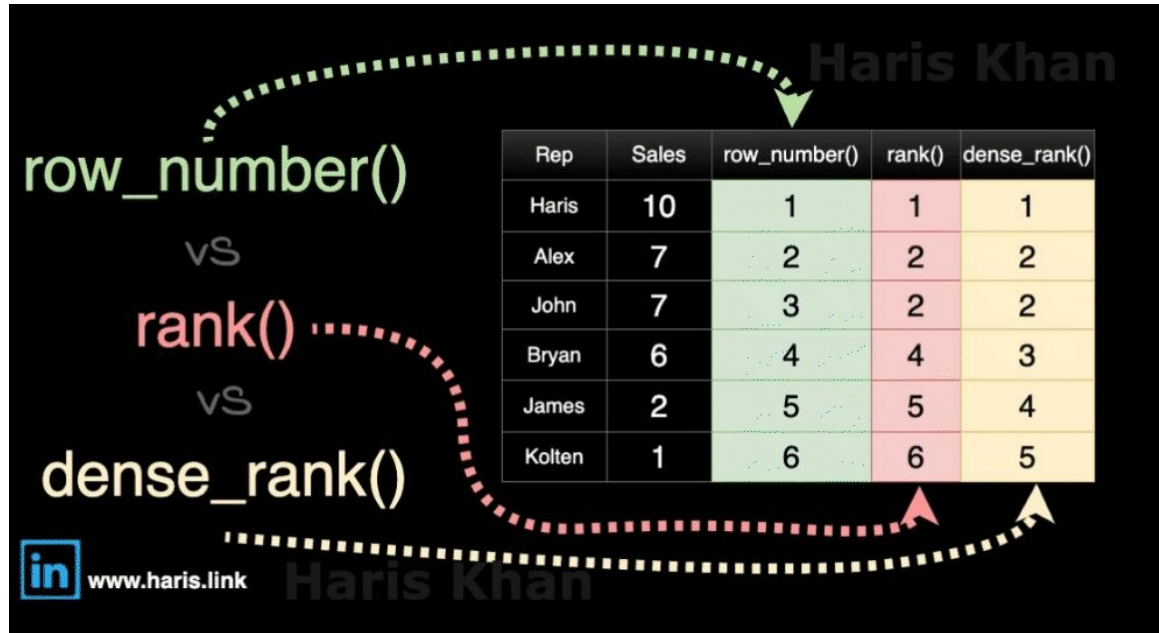
Outer query filters it.

```
SELECT course, avg_marks
FROM (
    SELECT course, AVG(marks) AS avg_marks
    FROM skill
    GROUP BY course
) AS course_avg
WHERE avg_marks > 75;
```

Windows Functions in SQL

row_number()
vs
rank()
vs
dense_rank()

Rep	Sales	row_number()	rank()	dense_rank()
Haris	10	1	1	1
Alex	7	2	2	2
John	7	3	2	2
Bryan	6	4	4	3
James	2	5	5	4
Kolten	1	6	6	5



 www.haris.link

